Lab 7

MSDS684_S70_Reinforcement Learning

Maryi Tatiana Palacios Giraldo

Regis University

Mayo 2026

Project Overview

This lab investigates the integration of planning and learning in reinforcement learning through the Dyna architecture, as described in Sutton & Barto (Chapter 8). The central question is how incorporating a learned model of the environment improves sample efficiency compared to purely model-free approaches such as Q-learning.

In standard Q-learning, the agent updates its value function solely from real environment interactions. This limits learning speed because each update requires new data, and value information propagates slowly across the state space. In contrast, model-based approaches learn an internal representation of environment dynamics and use it to generate simulated experience, allowing additional updates without further interaction.

In this lab, I implement Dyna-Q in the Taxi-v3 environment. The agent integrates three components: (1) direct reinforcement learning through Q-learning updates, (2) model learning via a deterministic dictionary mapping state-action pairs to observed transitions, and (3) planning through simulated updates sampled from the learned model. This structure reflects the Dyna architecture, where acting, learning, and planning occur simultaneously.

The hypothesis is that Dyna-Q will outperform standard Q-learning in terms of sample efficiency because each real step is supplemented with multiple simulated updates. Increasing the number of planning steps should accelerate convergence by propagating value estimates more quickly. However, this benefit depends on model accuracy.

To examine the limitations of model-based learning, the environment is extended with a dynamic change. In this setting, Dyna-Q may perform poorly because its model becomes outdated. To address this, I implement Dyna-Q+, which introduces an exploration bonus proportional to $\kappa\sqrt{\tau}$, encouraging the agent to revisit previously explored state-action pairs and update its model.

Finally, I implement prioritized sweeping, which improves planning efficiency by prioritizing updates with high temporal-difference error. This focuses computation on the most informative transitions and accelerates convergence.

Deliverables

GitHub Repository:

https://github.com/MPalacios-Analytics/DataScience_Projects/tree/630f43a4cb87fbbd49e4d2357e8a446f3be601bc/06_Reinforcement_Learning/Lab_7

s**Implementation Summary**

The implementation includes Q-learning, Dyna-Q, Dyna-Q+, and prioritized sweeping using the Taxi-v3 environment. The Q-table is represented using NumPy arrays corresponding to the discrete state and action spaces. A deterministic model is implemented using a Python dictionary mapping (state, action) pairs to observed (reward, next state) transitions.

Experiments were conducted over 500 episodes, with planning steps $n \in \{5, 10, 50\}$. Hyperparameters were set to $\alpha = 0.1$, $\gamma = 0.99$, and $\varepsilon = 0.1$ to balance learning stability and exploration. Dyna-Q+ incorporates an exploration bonus $\kappa\sqrt{\tau}$ with $\kappa = 0.001$ to encourage revisiting infrequently explored state-action pairs. Prioritized sweeping uses a priority queue based on temporal-difference error magnitude to focus updates on transitions with the highest expected learning impact.

Key Results & Analysis

The results demonstrate that Dyna-Q significantly improves learning speed compared to Q-learning by leveraging simulated experience. In standard Q-learning, each update depends on real environment interaction, limiting the rate at which value information propagates through the state space. In contrast, Dyna-Q performs additional updates using previously observed transitions stored in the model, effectively increasing the number of updates per real interaction and reducing sample complexity, as described in Sutton & Barto (Chapter 8).

Increasing the number of planning steps further accelerates convergence. With $n = 50$, the agent reaches optimal performance substantially faster than with $n = 5$ or $n = 10$. This occurs because more simulated updates allow faster propagation of value estimates across states. However, this comes with increased computational cost, illustrating the trade-off between computation and data efficiency.

In the non-stationary environment setting, standard Dyna-Q performance degrades due to reliance on an outdated model. Because the agent continues to simulate transitions that are no longer valid, incorrect updates are propagated through the value function. Dyna-Q+ mitigates this issue by introducing an exploration bonus $\kappa\sqrt{\tau}$, which increases the value of actions that have not been selected recently. This encourages exploration and enables the agent to update its model more quickly, resulting in faster adaptation to environmental changes.

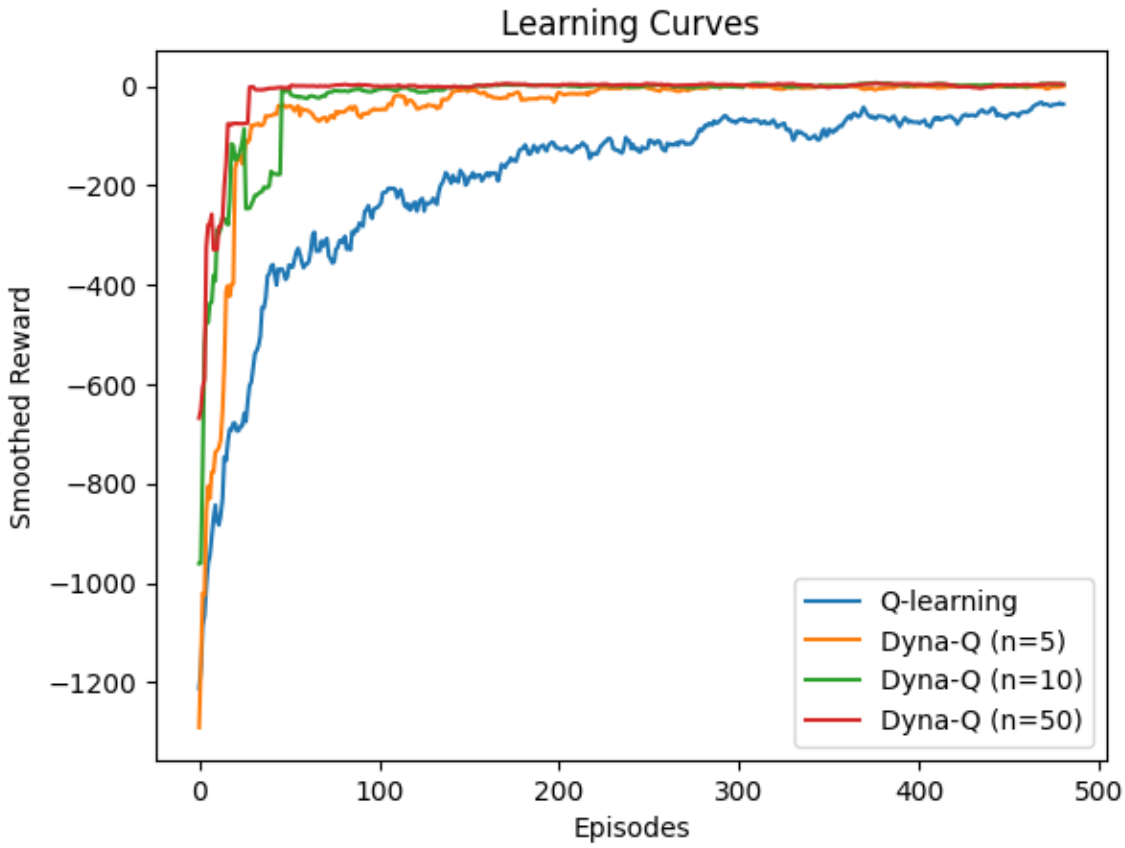
Prioritized sweeping further improves planning efficiency by focusing updates on state-action pairs with high temporal-difference error. Instead of sampling uniformly from the model, this approach prioritizes transitions that are expected to produce the

largest value updates. As a result, learning converges faster because computational effort is concentrated on the most informative experiences.

Overall, these results highlight that model-based methods can significantly improve sample efficiency, but their effectiveness depends on model accuracy. When the model becomes outdated or incorrect, performance can degrade unless mechanisms such as exploration bonuses are introduced.

Visualizations

Figure 1: Learning Curves



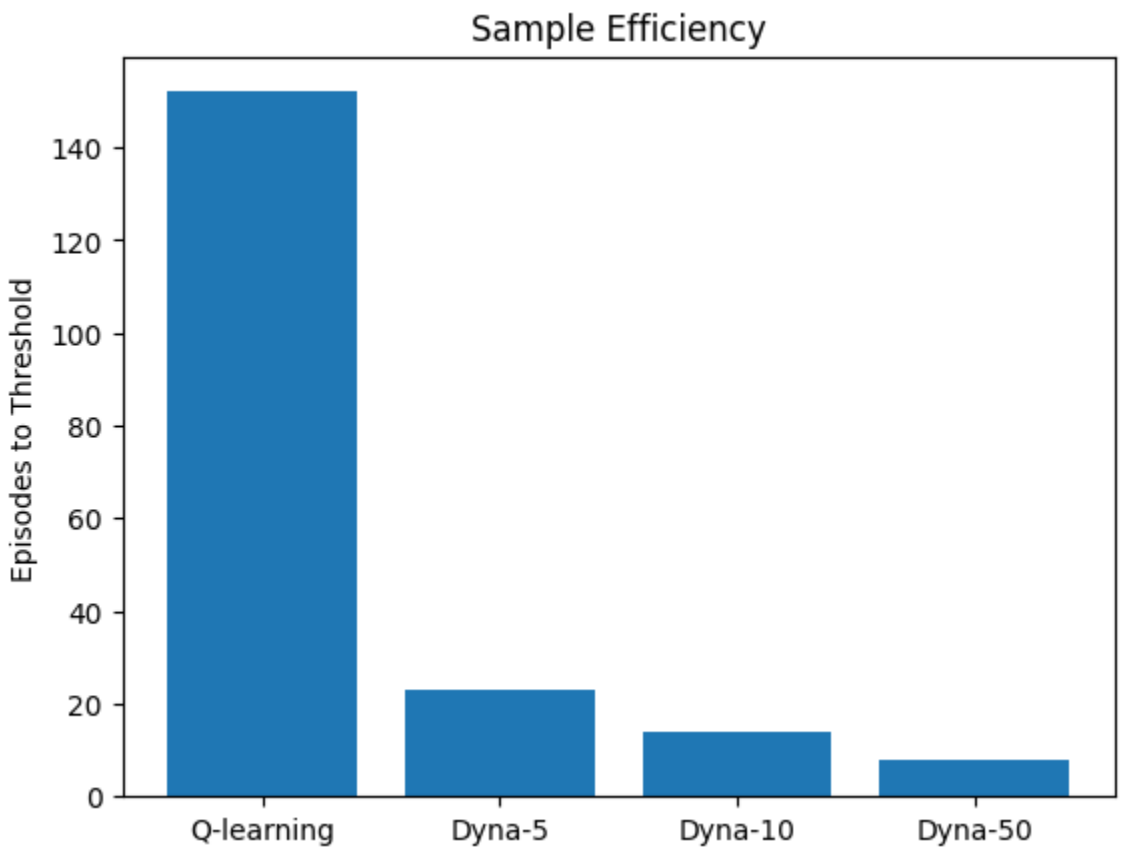
The learning curves show that Dyna-Q converges significantly faster than standard Q-learning, and that performance improves as the number of planning steps increases. While Q-learning updates the value function only from real environment interactions, Dyna-Q performs additional updates using simulated experience generated from the learned model.

This difference explains the observed behavior: Q-learning improves gradually because value information propagates slowly through the state space, whereas Dyna-Q

accelerates learning by performing multiple updates per real step. Increasing the number of planning steps (n = 5, 10, 50) further enhances this effect, with n = 50 achieving the fastest convergence.

This result aligns with Sutton & Barto (Chapter 8), where planning is described as a mechanism to increase the number of effective updates without requiring additional real data. In this case, Dyna-Q reduces sample complexity by leveraging previously observed transitions, allowing faster propagation of value estimates across states.

Figure 2: Sample Efficiency

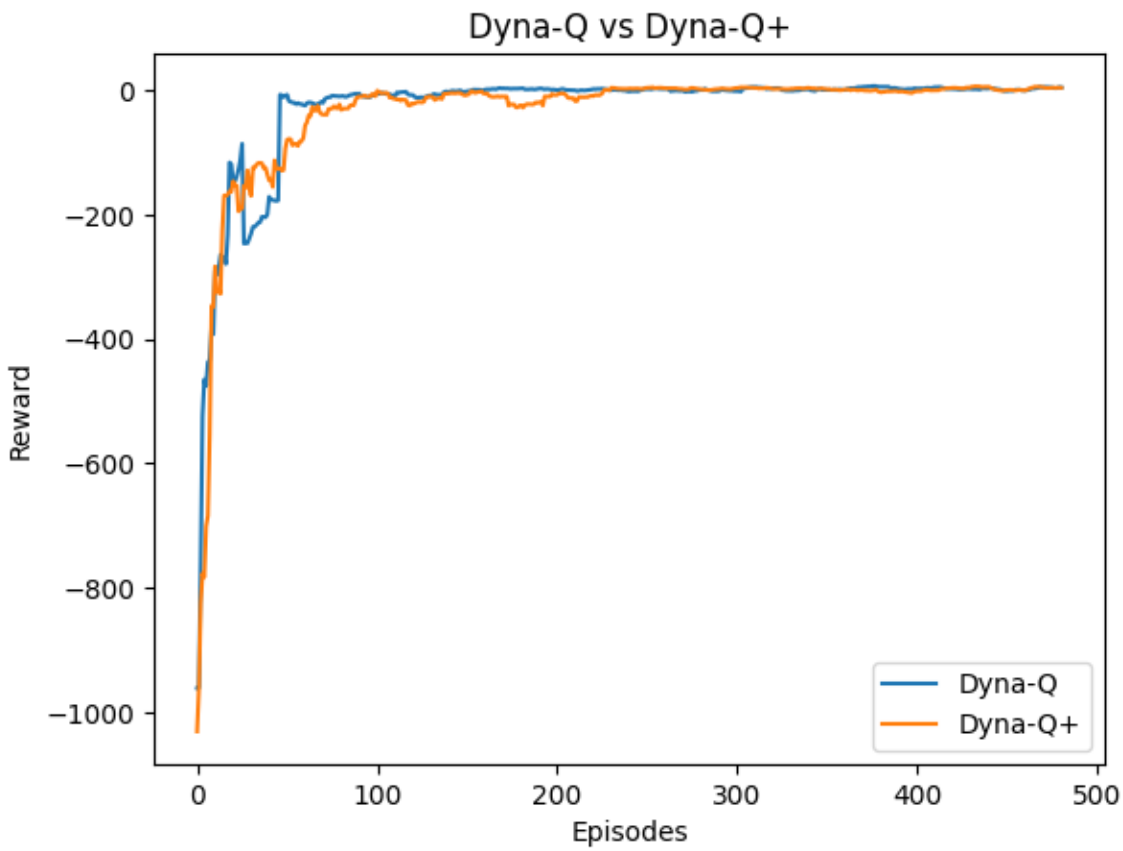


The sample efficiency results quantitatively confirm that Dyna-Q requires fewer episodes to reach a performance threshold compared to Q-learning. As the number of planning steps increases, the number of episodes required decreases substantially.

This occurs because each real interaction in Dyna-Q generates multiple simulated updates, effectively increasing the learning signal per step. In contrast, Q-learning relies solely on real transitions, making it less efficient in terms of data usage.

However, this improvement comes with a trade-off: increasing planning steps also increases computational cost. This highlights a key principle in reinforcement learning—there is a balance between sample efficiency (data usage) and computational efficiency (processing time). Dyna-Q improves the former at the expense of the latter.

Figure 3: Dyna-Q vs Dyna-Q+



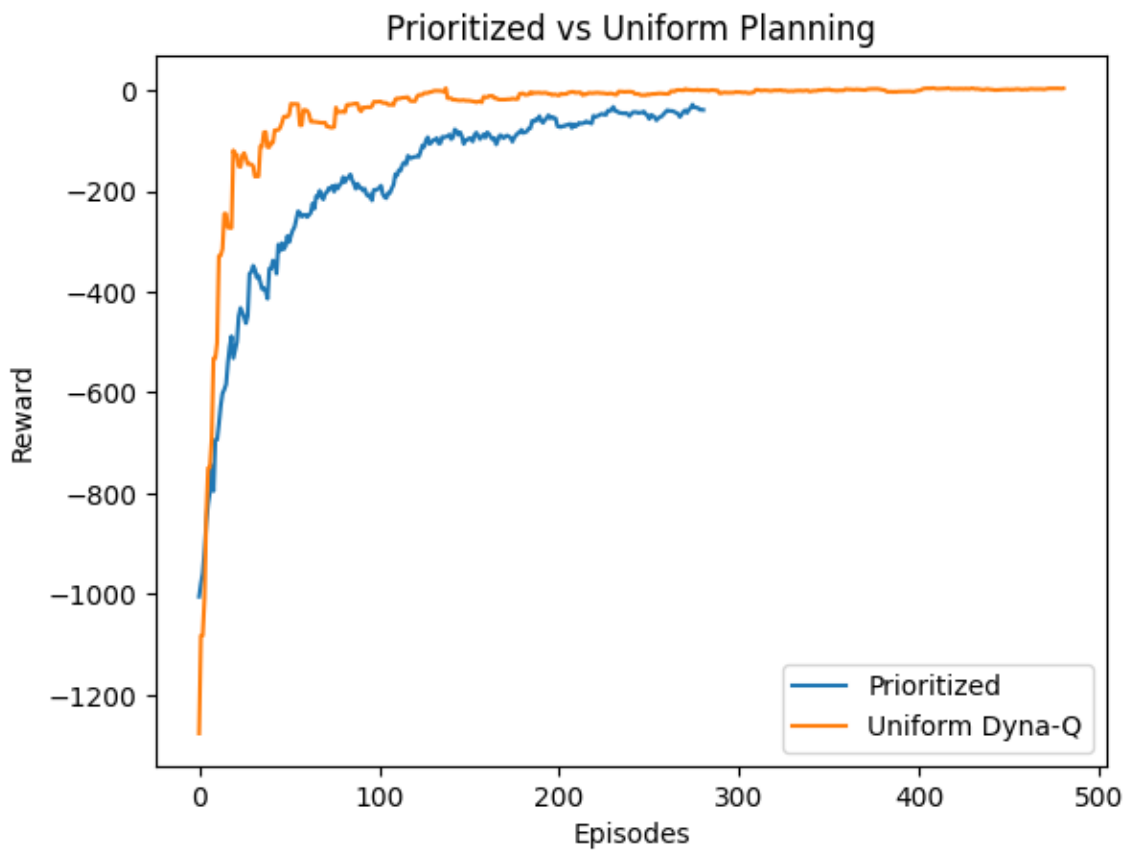
The comparison between Dyna-Q and Dyna-Q+ demonstrates the limitations of model-based learning in non-stationary environments. While both methods perform similarly in the early stages, Dyna-Q+ maintains more stable performance over time.

The underlying reason is that Dyna-Q relies on a learned model that assumes a stationary environment. When the environment changes, the stored transitions become outdated, leading to incorrect simulated updates that slow down adaptation.

Dyna-Q+ addresses this issue by introducing an exploration bonus proportional to $\kappa\sqrt{\tau}$, where τ represents the time since a state-action pair was last visited. This mechanism increases the value of less frequently explored actions, encouraging the agent to revisit them and update the model accordingly.

As a result, Dyna-Q+ adapts more effectively to changes, demonstrating that maintaining exploration is essential in environments where dynamics may shift over time.

Figure 4: Prioritized Sweeping



The results show that prioritized sweeping converges faster than uniform random planning. While standard Dyna-Q samples state-action pairs uniformly from the model, prioritized sweeping selects transitions based on the magnitude of their temporal-difference (TD) error.

This prioritization leads to more efficient learning because updates focus on transitions that are expected to have the greatest impact on the value function. Large TD errors indicate that current value estimates are inaccurate, so updating these transitions first accelerates the propagation of correct values through the state space.

This result highlights that not all experience is equally valuable. By focusing computational effort on the most informative updates, prioritized sweeping improves both convergence speed and planning efficiency.

AI Use Reflection

1. **Initial Interaction:**

I used AI to help design the structure of Dyna-Q, particularly how to integrate the model dictionary with Q-learning updates. My initial prompt asked how to efficiently store state-action transitions for planning. The AI suggested using a dictionary mapping (s, a) pairs to (r, s') and separating real experience updates from simulated planning updates.

2. **Iteration Cycle**

Iteration 1:

The initial implementation of Dyna-Q did not outperform Q-learning. I asked: "Why is my Dyna-Q not improving over Q-learning?" The AI suggested that model updates were not being properly incorporated into planning. I corrected this by ensuring the model was updated after every real step and used consistently during simulated updates.

Iteration 2:

Learning curves appeared unstable and inconsistent. I asked: "Why is Q-learning unstable in Taxi-v3?" The AI pointed out that the exploration rate was too high and planning steps were insufficient. I adjusted ε and increased the number of planning steps, which stabilized learning.

Iteration 3:

After introducing an environment change, Dyna-Q performance dropped significantly. I asked: "Why does performance degrade after environment modification?" The AI explained that the model became outdated. Based on this, I implemented Dyna-Q+ with an exploration bonus $\kappa\sqrt{\tau}$ to encourage revisiting less explored state-action pairs.

Critical Evaluation

While AI suggestions were helpful in identifying structural issues, they initially assumed a stationary environment and did not account for model degradation over time. This limitation required additional reasoning based on Sutton & Barto theory. I verified correctness by comparing learning curves before and after implementing fixes and ensuring that results aligned with theoretical expectations.

Learning Reflection

This lab clarified the trade-off between model-based and model-free reinforcement learning. I learned that planning improves sample efficiency by reusing experience, but it can introduce errors when the model is inaccurate. Implementing Dyna-Q+ reinforced the importance of continued exploration in non-stationary environments. Additionally, prioritized sweeping demonstrated how focusing updates on high-impact transitions can significantly improve learning efficiency.

Speaker Notes

- Problem: Compare model-free vs model-based RL using Dyna-Q in Taxi-v3
- Method: Q-learning combined with learned model and simulated planning updates
- Key idea: Planning increases sample efficiency by reusing past experience
- Result: Dyna-Q converges faster than Q-learning, especially with higher planning steps
- Challenge: Environment change revealed limitations of learned models
- Solution: Dyna-Q+ improved adaptability through exploration bonus
- Insight: Prioritized sweeping improves efficiency by focusing on high TD-error updates